WHITEPAPER

CERTIQS-KEY

Towards certifiable QKD key rates with AI-developed security proofs

Elias X. Huber[1,2,3], Becca Verghese[3], Matthieu Croci[3], Ian George[3], Michael Kasper[1,2], Marco Tomamichel[3]

[1]Fraunhofer Singapore Research Ltd., 1 Create Way, Singapore 138602
[2]Campus for Research Excellence and Technological Enterprise (CREATE), 1 Create Way, Singapore 138602
[3]Centre for Quantum Technologies, National University of Singapore, 3 Science Drive 2, Singapore 117543

24 October 2026 • CERTIQS-KEY v0.1.0 • github.com/fraunhofersg/certiqs-key

Abstract

Certifying a quantum key distribution (QKD) system requires a key rate backed by a complete security proof accounting for device imperfections and finite-size effects, with assumptions mapped to and tested on the real implementation. European cybersecurity agencies judged these requirements not yet met. This whitepaper presents CERTIQS-KEY, an open-source key rate engine built with these requirements in mind. It is aimed at QKD vendors and engineers, security evaluators and certification bodies, and security proof experts. CERTIQS-KEY computes fixed-length, finite-size key rates for discrete-variable protocols. Its modular architecture, inspired by OpenQKDSecurity, supports various device imperfections and strictly separates user inputs from the proof-critical backend. The user inputs specify the protocol and devices, with implemented examples for BB84, decoy-state BB84 and BBM92, and are kept as close as possible to the objects a QKD engineer actually works with. For instance, a receiver is described by its optical elements and detectors rather than by the finite-dimensional measurement operators (POVMs) a security proof requires, which only follow after a proof reduction such as squashing. CERTIQS-KEY outputs a key rate alongside a traceable *key rate report*, designed to answer the question an evaluator must decide: under what conditions is the claimed key rate secure for this real QKD system? It lists the security claim, the device model and inputs, and every assumption the result rests on. Underpinning this, the security proof is kept alongside the code that implements it and structured with future formal verification in mind: it is split into claims that form a dependency graph, from which the report's list of assumptions is computed. The key rates are rigorously bounded against floating-point imprecision end to end, using interval arithmetic. AI agents drafted, adversarially reviewed and implemented this proof and code under human supervision. We discuss what trusting AI-developed security proofs requires, the role that remains for human expertise, and implications for certification and protocol standards. CERTIQS-KEY is an early prototype: substantial human review is still pending.

Contents

1 Introduction **2**

2 Background **5**

2.1 Security in quantum key distribution 5
2.2 How a QKD protocol is modelled 6
2.3 Numerical key rates 6
3 CERTIQS-KEY and the agentic workflow 7
3.1 Architecture and software features 7
3.2 Workflow and agentic harness 11
3.3 Reviewing and extending the backend 13
4 Using CERTIQS-KEY 14
5 Discussion and vision 16
5.1 Trust and key rates 17
5.2 The role of human expertise in QKD security proofs 19
5.3 Implications for security certification 20
5.4 Standardisation 22
5.5 Limitations 25
5.6 Acknowledgement of prior work 25

1 Introduction

The deployment of quantum key distribution (QKD) in high-security environments comes with exceptional assurance demands. QKD promises information-theoretic security, yet in a 2024 joint position paper, the national cybersecurity agencies of Germany, France, the Netherlands and Sweden assessed QKD as 'not yet sufficiently mature from a security perspective' [1, pp. 1, 5]. Three of the reasons given concern the security proof: (1) no security proof for a practically relevant protocol, in a realistic model that accounts for device imperfections and finite key sizes, has been written up in a cohesive and comprehensive way, (2) no QKD protocol has been standardised with a matching proof that experts could scrutinise and (3) recognised evaluation criteria and methodologies, which would establish that a concrete device implements the protocol correctly and resists physical attacks, are still to be developed [1, pp. 4–5].

For assurance in real-world QKD deployments, we must be able to provide bounds on the key rate that are supported by proofs which address these concerns. These key rate calculations must also be *practical*, meaning the model we prove security for must include device imperfections and finite-size effects while resulting in a positive key rate when using currently existing optical devices over realistic distances. In recent years, modular numerical frameworks have become a promising avenue to provide practical key rates [2, 3], with open-source packages such as OpenQKDSecurity [4], which supports a broad class of device models and protocols, and ConicQKD [5, 6], which provides fast, general-purpose conic solvers for asymptotic and finite-size key rates.

However, these existing numerical frameworks remain primarily useful for a small group of experts rather than bringing the deployment of QKD into high-security environments. They only partially address the concerns of cybersecurity agencies, for a few reasons. First, the further development of these frameworks requires the incorporation of scattered theoretical and numerical results into self-contained security proofs with key rate calculations general enough to accommodate practical QKD implementations, see e.g. [7]. The development and modification of said frameworks is then limited simply by the small number of experts. Similarly, as made explicit in [8, Section 1.1], merely *using* these frameworks requires a background in quantum information theory, optimisation theory, and at least one programming language as well as understanding how these topics are unified for numerically modelling quantum key distribution. This places high barriers to using these tools for non-domain

experts including QKD hardware experts. Finally, ongoing security certification and standardisation efforts [9–11] specify the need for key rate claims to come with explicit assumptions that can be mapped to concrete evaluation activities. To the best of our knowledge, this is not implemented at the level of the software for any existing framework. This limits their relevance for certification and evaluation.

CERTIQS-KEY, the key rate calculation engine presented in this whitepaper, is a prototype that presents a potential path towards practical, traceable and self-contained key rate guarantees. This whitepaper is aimed at QKD vendors and hardware engineers, security evaluators and certification bodies, as well as security proof experts. CERTIQS-KEY started as an experiment based on the premise that publicly available frontier large language models (LLMs) can derive correct mathematical proofs, identify mistakes, and implement code to specification at a level comparable to, or exceeding, that of human domain experts. We set up an AI-driven draft-and-verify workflow and agentic harness to generalise and expand existing work under human supervision (Section 5.6). This workflow resulted in a self-contained repository that includes a software implementation of the key rate calculation along with the mathematical derivations and assumptions that justify it. More concretely, the repository is composed of seven modules (see Figure 1), each module containing a software implementation and a *contract* document of its underlying mathematical *claims*. A claim links to the code that implements it, the underlying assumptions and dependencies, as well as the proof that justifies it (see Figure 2). In this manner the security proof, the numerical method, and the assumptions that underpin them are explicit and traceable. CERTIQS-KEY aims to be accessible to the broader community without compromising rigour. For this reason, two layers are kept strictly separate:

- **INTERFACE**: The inputs and the key rate report, for people who interact with the software solely to input a description of their protocol and receive reports about the key rate and guarantees of their protocol.
- **PROOF BACKEND**: The security proof and the key rate calculation itself, for people who may further develop both new and existing tools for establishing security of protocols.

In this whitepaper, we introduce how CERTIQS-KEY is designed and developed and how its backend can be reviewed and extended (Section 3), how it can be used to compute key rates (Section 4), and initiate a discussion with the broader QKD community on the role of AI-driven numerical key rate frameworks and implications for certification and standards (Section 5). Below, we summarise which parts of this document we believe to be relevant to key rate users, evaluators and security proof experts.

For QKD vendors, users, and experimentalists (**INTERFACE**), Section 3.1 elaborates on the software architecture and features, while Section 4 discusses using CERTIQS-KEY for practical key rate calculations. The interface of CERTIQS-KEY provides a practical entry point that is largely free of proof-specific formalism and reduces the information users must supply to the specification of protocols, device models, and run parameters. For example, the software automatically compiles an input description of a receiver constructed out of linear optical elements and threshold detectors into the mathematical objects contemporary numerical security proofs require. To further lower the barrier to entry, CERTIQS-KEY ships a Model Context Protocol (MCP) server. This allows any MCP-capable AI coding agent, such as Claude Code, Codex, or Cursor, to edit protocol parameters, create new protocol instances, and run key rate calculations. Every result comes with a key rate report (see Figures 1 and 9) which contains the security statement, the inputs, and every assumption the result rests on. This allows confidence that the key rates match the real QKD system even when using a workflow where it is an AI agent that formalises the protocol. For this, the human user has to

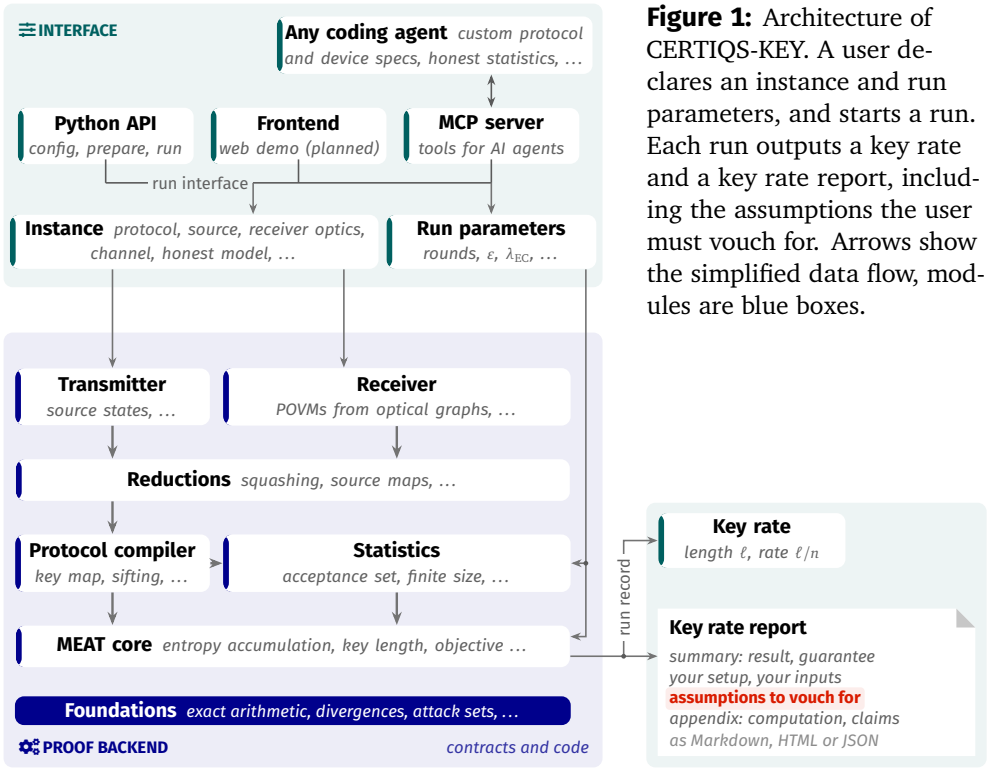


Figure 1: Architecture of CERTIQS-KEY. A user declares an instance and run parameters, and starts a run. Each run outputs a key rate and a key rate report, including the assumptions the user must vouch for. Arrows show the simplified data flow, modules are blue boxes.

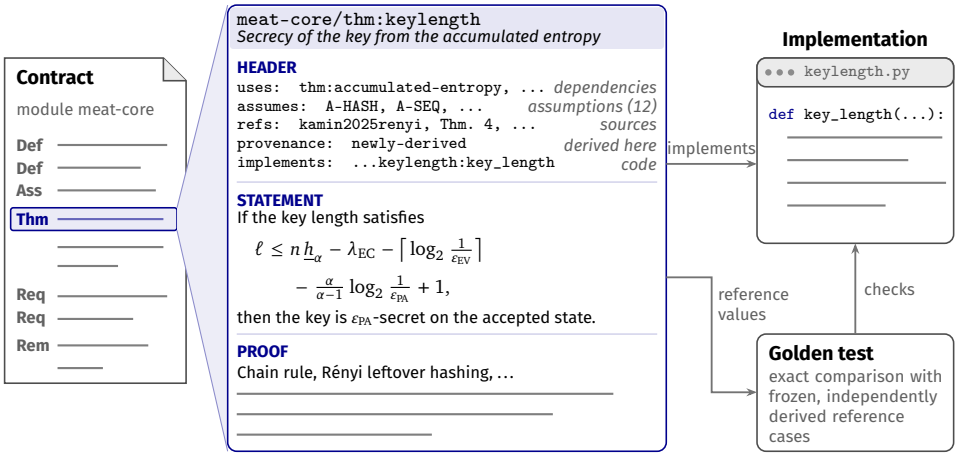


Figure 2: PROOF BACKEND Contract and implementation, with a sample claim enlarged: the header contains the function name of the implemented code. A test then checks the implemented code against reference values derived from the claim's statement.

review the device model and assumptions stated in the key rate report to vouch that each listed assumption holds for the actual device.

For QKD security evaluators and cybersecurity agencies, CERTIQS-KEY by design delivers traceable key rate reports. The design goal of these reports is to answer everything an evaluator might ask when deliberating: 'Under what conditions is the claimed key rate secure for this real QKD system?' In practice, at least two concerns remain. The first is trust in the underlying AI-written proof and code to be correct in the first place. As we discuss in Section 5.1, beyond *trust by process* today, we believe that formal verification offers a promising future avenue. We also discuss in Section 5.3 how reducing the human review surface may make certified QKD key rate software viable and alleviate challenges from currently insufficiently available security proof expertise. The second is linking formally

stated assumptions and modelling decisions to realistic QKD devices. Here, we point to some standardisation gaps and opportunities in Section 5.4.

For theoretical security proof experts (⚙ PROOF BACKEND), we expect the agentic workflow and how to review and extend the backend (Sections 3.2 and 3.3) as well as the discussions on trust and the role of human expertise (Sections 5.1 and 5.2) to be of timely interest. CERTIQS-KEY contains some novel theoretical contributions of independent interest.[1] Among these, we highlight the certification against numerical and solver imprecisions through a posteriori certification and interval arithmetic (see Section 3.1, 'Numerical rigour'). To the best of our knowledge, CERTIQS-KEY is the first finite-size QKD key rate calculation engine that certifies its key length against floating-point imprecision end to end. That said, we stress that at the time of publication of this whitepaper, significant human review of these results is still pending and the contracts that state them are not very easy to parse for a human reader. We intend to publish a follow-up technical paper with more details in the future.

Code availability The code for CERTIQS-KEY and the full mathematical derivations are publicly available on its GitHub repository.

2 Background

This section introduces some basic background and QKD security proof terminology used in the subsequent sections. Readers familiar with numerical QKD security proofs may wish to skip this section. We refer the reader to Tupkary et al. [7] for more extensive background.

2.1 Security in quantum key distribution

The goal of (fixed-length) quantum key distribution is to output a secret key. A secret key is uniformly random, perfectly correlated classical data shared between two users, Alice and Bob, that is independent of all other physical systems which could be available to an eavesdropper, Eve. This goal is achieved by the QKD protocol approximating an ideal process that either aborts or outputs a secret key of fixed length up to a tolerated error, or security parameter, $\varepsilon \in (0, 1)$. As explained in detail in [7, 12], this results in requiring that the output joint state ρ of a QKD protocol always satisfies the equation

$$\underbrace{\Pr[\Omega_{\text{acc}}]}_{\substack{\textbf{weight: how often}\\ \text{the execution}\\ \text{does not abort}}} \cdot \overbrace{\frac{1}{2}\left\| \underbrace{\rho_{K_A K_B E \,|\, \Omega_{\text{acc}}}}_{\substack{\textbf{real output:}\\ \text{the keys and}\\ \text{everything Eve}\\ \text{holds conditioned}\\ \text{on acceptance}}} - \underbrace{\tau_{K_A K_B} \otimes \rho_{E \,|\, \Omega_{\text{acc}}}}_{\substack{\textbf{ideal output: uniform,}\\ \text{correlated data } (\tau),\\ \text{independent of } (\otimes)\\ \text{everything Eve holds}}} \right\|_1}^{\substack{\textbf{trace distance: the bias between}\\ \text{distinguishing the real and ideal output}}} \leq \underbrace{\varepsilon}_{\substack{\textbf{security}\\ \textbf{parameter:}\\ \text{small, fixed}\\ \text{in advance}}} \tag{1}$$

where K_A and K_B are registers storing Alice and Bob's keys of ℓ bits, E is a register storing all information available to an adversary, and Ω_{acc} denotes the event that the protocol does not abort.

In CERTIQS-KEY, assuming perfectly authenticated classical channels, the security parameter decomposes:

$$\varepsilon = \varepsilon_{\text{PA}} + \varepsilon_{\text{EV}} \,. \tag{2}$$

[1] Including squashing with constraints, POVM modelling, and solvers of the underlying convex optimisation problem.

Here $\varepsilon_{\mathrm{PA}}$ and $\varepsilon_{\mathrm{EV}}$ are identified as the secrecy and correctness parameters, respectively. The subscripts denote the steps of the protocol as described in Section 2.2. The formulation of security in (2) is a standard result, cf. [7, Thm. 2.1].

A QKD security proof essentially derives a formula for the key length ℓ, proving that whenever the key length used by the QKD system is equal to or less than this formula, the security condition (1) holds for the resulting key. (Assuming the correct privacy amplification was implemented, see next subsection.)

2.2 How a QKD protocol is modelled

A QKD implementation performs a sequence of steps, including quantum measurements and classical post-processing, before outputting a key to Alice and Bob. In order for a numerical framework to apply to a given QKD implementation, some protocol structure has to be assumed. Unfortunately, to date, no international QKD protocol standard prescribes a fixed protocol structure, and practical implementations can differ. The United Nations' International Telecommunication Union's recommendation on the framework of QKD protocols in QKD networks, ITU-T X.1711 [13], presents a framework of QKD protocols and is likely the closest document to a published international QKD protocol standard at the time of publishing this whitepaper. The security-proof literature describes a similar generic structure [7, Sec. 3]. CERTIQS-KEY is compatible with both and supports generic entanglement-based (EB) and prepare-and-measure (P&M) protocols with the structure shown in Figure 3.

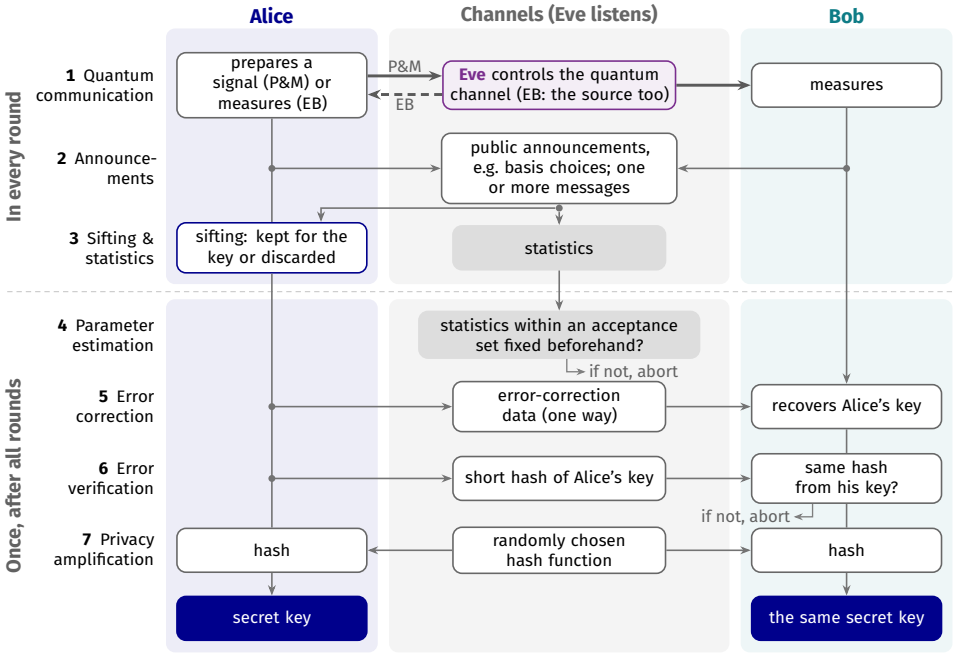


Figure 3: The protocol structure CERTIQS-KEY accepts. The middle lane holds what is public: messages (white) and checks anyone can compute from them (grey); a step that uses a party's own data sits in that party's lane. The figure shows direct reconciliation (Bob error-corrects to Alice's key); CERTIQS-KEY also accepts reverse reconciliation.

2.3 Numerical key rates

In textbooks and introductory quantum information courses, QKD is often presented in significantly idealised settings that are further simplified. These idealisations and simplifications include: the number of sent signals tending towards infinity ('asymptotic rate'), Eve's attack

being identical per round ('collective attacks'), and perfect single-photon sources and detectors. Often this results in an asymptotic key rate that is easy to compute and is a function of what is called the 'quantum bit error rate' (QBER). However, all of these assumptions introduce significant gaps between what is implemented in the real world and the model of the protocol for which security is established.

With numerical key rates, these gaps can be largely resolved. First, security proofs can do finite-size analysis, which takes into account that the number of signals is finite and the statistical fluctuations that come with this. Second, security proofs can handle security against 'coherent attacks', which means Eve can manipulate the signals arbitrarily, only bound by the laws of quantum physics. Finally, one can more accurately model the non-ideal optical devices used in practice. While there exist special cases where all of these relaxations can allow for analytic security proofs using only a few more parameters than QBER, e.g. [14], generally it becomes intractable to analytically prove security while obtaining near-optimal rates. This holds true quite generically both in the asymptotic [15, 16] and finite-size [17] regimes, which limits the practicality of analytic security proofs. For this reason, numerical methods for proving security of QKD [2, 3, 6, 17, 18] have become critical.

All of these numerical methods then follow the same high-level structure:

1. privacy amplification results in a key that is $\varepsilon_{\mathrm{PA}}$-secret (the secrecy part of the security criterion of Section 2.1) so long as the output key length ℓ is bounded above by a function proportional to a worst-case entropic quantity, and
2. this worst-case entropic quantity can be reduced to an optimisation problem in terms of the single-round description (upper band of Figure 3) of the QKD protocol.

Once these steps have been established, the main issue is guaranteeing that numerical methods provide a true lower bound on this optimisation problem. That numerical methods will provide a true lower bound is not a priori guaranteed [2, 15]. However, if a true lower bound is guaranteed, then the output of the numerical security proof can be used to provide guarantees on the implemented protocol. It is upon this existing numerical approach that CERTIQS-KEY is built.

3 CERTIQS-KEY and the agentic workflow

3.1 Architecture and software features

This subsection describes what CERTIQS-KEY is designed to do and summarises what its module contracts state. CERTIQS-KEY computes fixed-length, finite-size key rates based on the marginal-constrained entropy accumulation theorem (MEAT) [19]. Its architecture is modular, self-contained, and traceable by design, aiming to provide accessible and broadly applicable key rate calculations.

Module structure and user↔backend separation CERTIQS-KEY is separated into seven modules as seen in Figure 1. Each consists of a contract, holding mathematical claims, and the code that implements it.[2] Two modules describe the devices: the transmitter module models Alice's source, and the receiver module turns a description of passive optics and threshold detectors into the measurement (POVM) it performs. The reductions module replaces these physical, infinite-dimensional devices with finite-dimensional models that may decrease the resulting key length but never compromise security. The protocol compiler combines the reduced devices with the protocol's announcements, sifting and key map into one finite

[2] At this release, the contracts hold 815 claims (definitions, assumptions, proof statements and remarks) in about 67,000 lines of LaTeX, the code about 141,000 lines and the tests about 219,000 lines.

description, and the statistics module builds the acceptance test (the parameter estimation step of Figure 3) from the expected honest behaviour. The MEAT core turns the finite protocol description and the acceptance test into a key length. (The foundations module provides the number model and the quantum-information tools the others share.) Submodules, such as decoy states, detector imperfections or one particular squasher, are self-contained additions that a protocol uses only where it needs them.

We intentionally designed CERTIQS-KEY so that a key rate user never (accidentally) edits proof-critical code. A user of CERTIQS-KEY provides two things: an instance and the run parameters. The instance declares the protocol, source, receivers, channel, and honest model. The run parameters include the number of rounds, the security parameters, and the error-correction leakage λ_{EC}. CERTIQS-KEY then provides a Python interface and an MCP server (explained in Section 4) to pass these specifications into the key rate calculation. Specifying the protocol is therefore meant to be an input step that does not require editing the proof-critical backend. The CERTIQS-KEY backend, prior to a run, compiles the instance into a finite description the MEAT core module reads: a short list of systems, measurements, announcements and constraints, and checks a fixed set of admissibility conditions on it (Figure 12). This is different from existing codebases where, even if the user were not to change the equivalent of the MEAT core module, the functions of the protocol compiler, transmitter, receiver and reductions modules would have to be performed by hand prior to obtaining key rates.

CERTIQS-KEY is built with future formal verification in mind. The claim files of the module contracts provide an informal blueprint in the style of past Lean formalisation projects [20]. Each already has a (currently empty) header field for a formal counterpart (e.g. a Lean formalisation). The structure is meant to stay largely unchanged once formalisation begins, which would shrink what a human has to review to the interfaces and the key rate report (Section 5.3).

Features The goal of CERTIQS-KEY is to provide an accessible interface (see use-flow in Figure 8) general enough to accommodate generic discrete-variable protocols such that a key rate user never needs to edit proof-critical backend code and mathematics. It is unlikely that the interface already fully lives up to this ambition. The current contracts and code for the receiver, transmitter, reductions and protocol compiler modules would likely require modifications for protocols that significantly differ from those already integrated into CERTIQS-KEY, while the MEAT core module is protocol agnostic within the limits of its input format (Figure 12).

That said, we have already incorporated BBM92, BB84 and decoy-state BB84 with weak coherent pulses into CERTIQS-KEY. In all cases, the receiver was either modelled as perfect or made up of generic passive optical elements and threshold detectors similar to Figure 4. The deep dive into the receiver module below provides more details on what measurement models CERTIQS-KEY currently supports.

Module deep dive: the receiver

For a concrete example, we consider the receiver module in more detail. The receiver is one place where a vendor's device data would enter the proof. The general structure of the receiver module is the same as all other modules: declared inputs, outputs with stated guarantees, and a list of assumptions required for the outputs and their guarantees to hold.

Inputs: Instead of having to supply the matrices ('POVMs') that model the quantum

measurements as inputs, CERTIQS-KEY allows users to input a graph of the passive linear optics and threshold detectors on the receiver's side. For the passive linear optics, the supported *catalogue* is: beamsplitters, polarising beamsplitters, wave plates, rotators, mirrors, phase shifters, loss, polarisers, mode routing, and user-provided custom elements. Each threshold detector must also be specified in terms of its efficiency and a dark-click probability. For example, for our implementation of BBM92 (Figure 4), the input is a graph of a beamsplitter, a half-wave plate, two polarising beamsplitters, and four threshold detectors. Alternatively, a user who has characterised a device can input a custom matrix representing its linear optics (building on Kamin and Lütkenhaus [21, Sec. VII D]).

Outputs: Extending the mode-transformation description and multiclick bounds of Kamin and Lütkenhaus [21, Sec. VII] to a general framework, the receiver module computes the POVM elements for each click pattern (no click, a single click in H, V, D or A, or several clicks) on every photon-number block up to a specified cutoff N. Above the cutoff, it calculates a lower bound on the probability that several detectors click, which is needed in the reductions module to cut off at a lower photon number without compromising security (through constructions called *flag-state squashers* [22]).

Due to the modular structure of CERTIQS-KEY, a knowledgeable user could in principle bypass this module, do the receiver POVM calculations and reductions themselves, and then directly start at the protocol compiler module.

Assumptions: The output of the receiver module rests on multiple assumptions which a user of CERTIQS-KEY must vouch for, among them:

- **Trusted receiver**: the declared receiver describes the physical one exactly and is under the party's control;
- **Exact characterisation**: every parameter is exact, without an error bar (a problematic current limitation, see Section 5.5);
- **Passive optics**: the optics are passive and linear;
- **Declared optical modes**: every optical path into a detector is a declared mode (which, for example, wavelength-shifted attacks violate in practice);
- **Ideal threshold detectors**: apart from detection efficiency and dark count, the detectors are ideal threshold detectors, without dead time, afterpulsing or timing jitter.

Each of these assumptions appears in the key rate report every time the receiver module is used.

For a concrete example, Figure 6 shows key rates for BBM92 (entanglement-based QKD). To calculate key rates for BBM92, the BBM92 *instance*, containing everything specific to the protocol, is specified as input to CERTIQS-KEY: an untrusted source emits photon pairs, and each party measures one photon in a passive receiver of four threshold detectors.

With every run, CERTIQS-KEY creates a run record and a key rate report (Figure 9). The key rate report lists every assumption beneath the result, automatically computed from the declared dependencies of the claims used in the run (Section 3.2, Figure 2). In addition, the report documents the device model (including the receiver and transmitter, but also the protocol's processing steps) and all input parameters for the run. The key rate report also differentiates what the user inputs from engine-derived results, and links each statement to its place in the module contracts. This clearly documents the user responsibilities, which are to vouch that the listed assumptions hold for the actual devices.

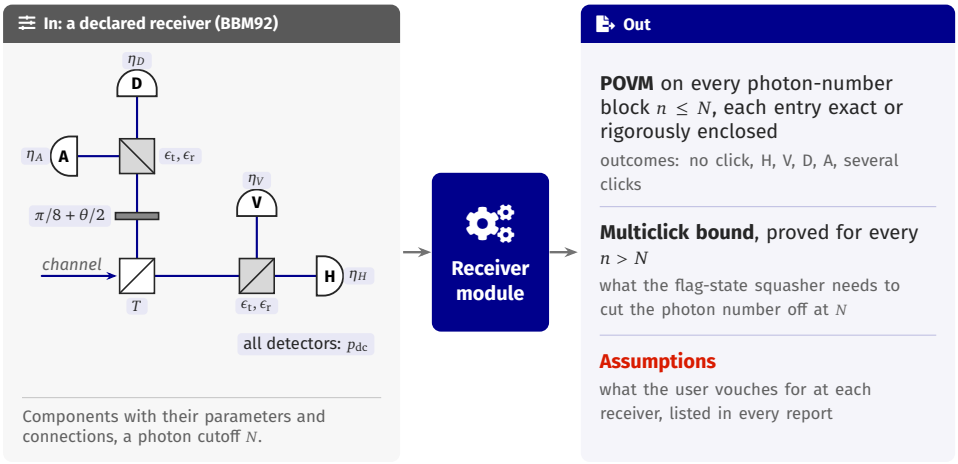


Figure 4: PROOF BACKEND The receiver module applied to one receiver of the BBM92 example. Left: the input, a receiver declared by its optics, with symbols of its parameters (for Alice, the analyser rotation $\theta = 0$). Right: the output of the module.

Numerical rigour PROOF BACKEND Our framework for numerical key rates rests on prior work that made such calculations reliable in the first place. Winick et al. [2, Fig. 1] split the calculation into two steps. An optimiser first attempts to find a near-optimal attack. The convex objective is then linearised at that point, and the dual of the linearised problem turns the result into a reliable lower bound, however good or poor the first step was. CERTIQS-KEY builds on this idea.

While these methods in theory guarantee that the reported key rates are lower bounds of the true key rate, this is not necessarily the case in practice as the optimisation itself is computed in floating point. Worse, small deviations (such as from rounding) in the device model can have a large impact on the key rate from the MEAT formula.[3] CERTIQS-KEY labels every reported number with one of three tiers, in increasing reliability:

- An *estimate* is the optimiser's value, which claims neither optimality nor a bound ('minimise and hope for the best').
- A *first-order* number is a lower bound that is rigorous for the formulas proven in the contracts, but evaluated in floating point, so no claim bounds its rounding errors. In our view, while some prior work partially accounted for numerical imprecision [2, 23], it remained at this first-order tier, or certified only the asymptotic entropy bound [24].
- A *certified* number is that same bound computed again with every arithmetic step, from the declared data to the key length, exact or rigorously enclosed in rational intervals or ball arithmetic. Here, the term 'certified' does not mean certification by an evaluation body, but rather comes from the idea of a posteriori certification [25], i.e., *search fast (but without guarantees), certify after*. This is the tier CERTIQS-KEY implements by default.

[3] To see why such an impact can occur, note the MEAT objective contains a relative-entropy term, $\frac{\alpha}{\alpha-1} D(\mathbf{q} \,\|\, \Phi[\omega])$ where $\mathbf{q}$ is a frequency distribution the acceptance test will accept and $\Phi[\omega]$ is the distribution on measurement outcomes on input state ω (see [3, Thm. 6] where it is written $D(\mathbf{q} \,\|\, \nu_{\hat{C}}^{\rho})$). The relative entropy can be written as $\sum_x \mathbf{q}(x) \log_2(\mathbf{q}(x)/\Phi[\omega](x))$. If the user defines an acceptance set and measurement such that there is $\hat{x}$ where $\mathbf{q}(\hat{x}) \gg \Phi[\omega](\hat{x})$ for all inputs ω, then the relative entropy term becomes large. Typically, this is only the case when the statistics $\mathbf{q}$ in the acceptance set are not achievable by the device model statistics $\Phi[\omega]$. However, if $\Phi[\omega](\hat{x})$ is very small while $\mathbf{q}(\hat{x}) > 0$, then small (in absolute terms) changes in $\Phi[\omega](\hat{x})$, such as from rounding the measurement, can result in large changes of $\frac{\alpha}{\alpha-1} D(\mathbf{q} \,\|\, \Phi[\omega])$. Mathematically, the reason is that, with $q = \mathbf{q}(\hat{x})$ and $p = \Phi[\omega](\hat{x})$, the derivative $\frac{\partial}{\partial p}\left[q \log_2 \frac{q}{p}\right] = -\frac{q}{p \ln 2}$ is unbounded as $p \to 0$. In summary, without further guarantees on the acceptance set and the measurement, the relative entropy term in the optimisation can grow without bound.

The optimiser runs in fast floating point, and only a few points that determine the key length are recomputed rigorously so that the recomputation takes only a small fraction of the run's computational cost.

While a posteriori certification has been used for numerical optimisation in other fields [26] and fundamentally uses the same ideas as the existing framework by Winick et al. [2], we highlight it in this section for two reasons:

1. As far as we are aware, CERTIQS-KEY is the first finite-size QKD key rate calculation engine that certifies its key length against floating-point imprecision end to end.
2. Connecting to the subsequent Section 3.2 on the workflow and agentic harness, the full numerical certification demonstrates how AI-driven workflows can in some cases deliver additional rigour over prior work by patiently doing the tedious work (here: interval arithmetic and error propagation). This work would be time-intensive for human experts and, despite being an important step for QKD security assurance, is likely not sufficiently rewarded, academically or commercially.

We emphasise, as in the introduction, that these results are very preliminary and pending human review. For readers familiar with Winick et al. [2], Figure 5 gives a high-level overview of the steps involved in CERTIQS-KEY's a posteriori certification.

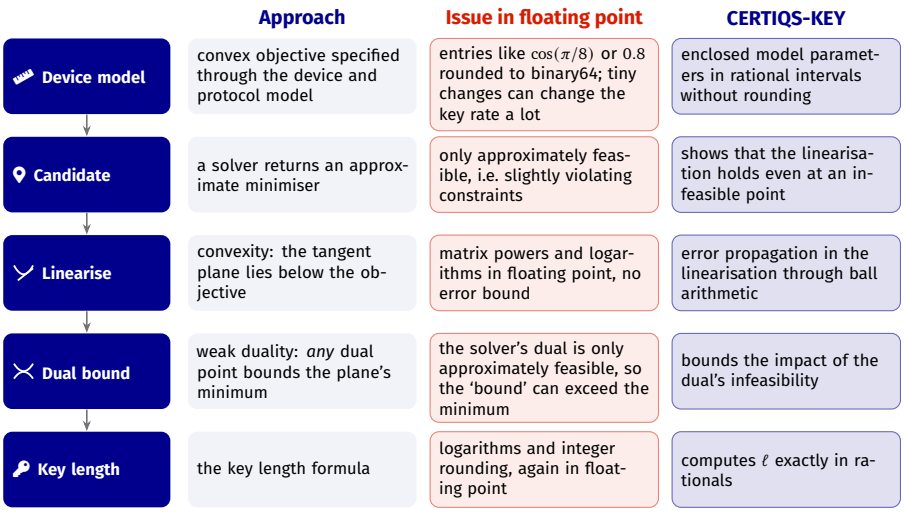


Figure 5: PROOF BACKEND Sketch of the certified path. Approach: the steps of Winick et al. [2]. Issue: where floating point can push each step above the true key rate. CERTIQS-KEY: how its certified evaluation answers each issue. Certification covers the arithmetic only: the modelling assumptions remain, and so does the code implementation review, including that of the ball-arithmetic library used [27].

CERTIQS-KEY also exploits the observation that no trust is placed in the convex solver itself. The convex solver must propose candidate points, and a poor solver can loosen the bound but cannot raise it above the true value. As a consequence, the solver can be replaced or aggressively optimised, including by AI agents, without touching the security argument.[4]

3.2 Workflow and agentic harness PROOF BACKEND

A QKD security proof is usually written as a paper, with the code that evaluates its key rate written afterwards as a companion. CERTIQS-KEY keeps mathematics and code in one

[4] Besides plain Frank–Wolfe [28], CERTIQS-KEY offers several other search methods. The default runs several of them in turn and keeps the best bound. In the BBM92 example of Section 4, it runs in mere seconds, while Frank–Wolfe stalls and, after about 8 to 9 minutes, still falls short of the default run's key by a large margin.

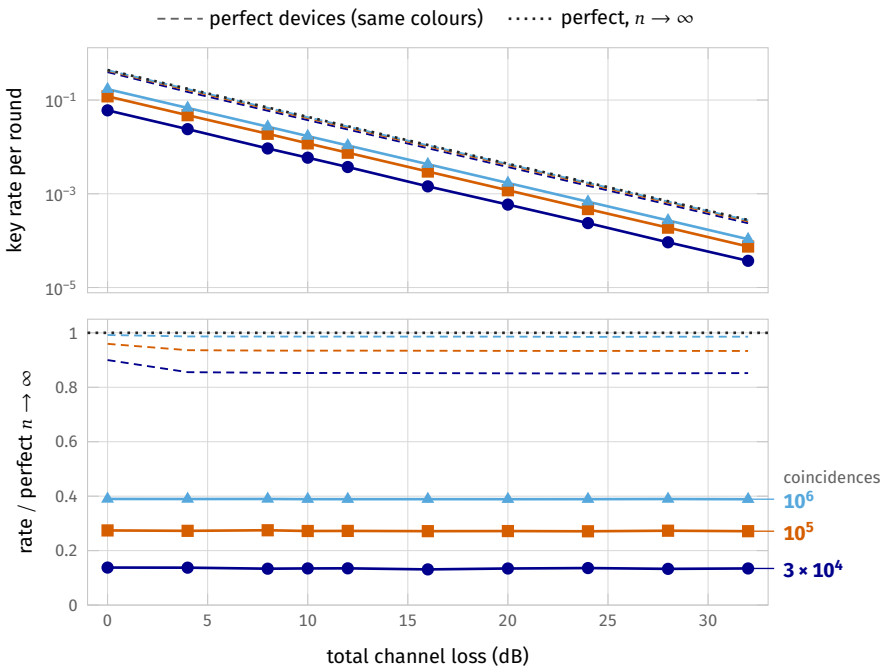


Figure 6: INTERFACE Key rate of BBM92 against channel loss, from the example setup of Section 4. Each colour is a block size, the number of rounds chosen so that 3×10^4, 10^5 or 10^6 coincidences are expected (as in Figure 10). Solid lines are realistic devices, dashed lines perfect ones at the same channel, and the dotted line the perfect devices' rate for infinitely many rounds. Bottom: each rate relative to that limit. The share reached hardly changes with the loss. The asymptotic rate per round is $(1-\gamma)\,\eta\,\frac{1}{2}\left[1-(1+f_{\mathrm{EC}})\,h(Q)\right]$, with test fraction $\gamma = 1/8$, transmittance η, $\frac{1}{2}$ for matching bases, error-correction efficiency f_{EC}, binary entropy h and bit error rate Q (zero for perfect devices).

repository and handles the mathematics in a similar way to writing code: each definition, assumption, lemma, theorem or requirement on the implementation is a short file of its own, a *claim*, whose header declares the claims it uses, the assumptions it rests on, its sources with locators, and the code that implements it (Figure 2). The module contract documents are assembled from these files. Because every dependency is declared, tools (scripts) compute what a reader of a paper tracks in their head: the assumptions beneath a result, the claims to re-review when one changes, and whether code exists for each claim marked as implemented. A claim is only as reviewed as the least-reviewed claim beneath it. Deriving thus takes on the habits of software development: version control, dependency tracking, automated checks and tests.

The development of the repository proceeds in draft-and-verify loops using various AI sub-agents with distinct instructions:

- A *drafter agent* writes or amends a batch of claims and proofs.
- A *verifier agent* reviews the batch adversarially. It sees only the resulting files of the drafter. It can neither edit the files nor see the drafter's reasoning. It grades its findings in the files by severity.
- An *orchestrating agent* runs loops of drafter and verifier agents until only minor findings remain. The orchestrating agent judges which minor findings are acceptable, and escalates what it cannot resolve to the human operator. If convergence to minor findings is not reached after three iterations of verifier and drafter, the orchestrator escalates to the human operator.
- The same draft-and-verify loop then runs for the code: an *implementer agent* writes code and tests from the claims, and an *implementation reviewer agent* checks them

against the claims.
- The *human operator* accepts each batch, first of claims and then of code, and makes the judgement calls escalated to them.

To support the LLMs, tools (scripts) check form (e.g. cross-references resolve), generate views and contract documents, and help with mechanical tasks such as checking dependencies or showing version changes. This design grew out of trial and error. It was inspired by works such as the blueprints of formalisation projects [20], the workflow of the agent-driven formalisation of Fermat's Last Theorem [29] and the Prove2Me platform it used [30], and the rapidly evolving capabilities of coding agents. Many of the lessons learned are documented in CERTIQS-KEY's guiding principles.

The human acceptance steps are intended to keep some human review and control over the direction. We experimented with different review surfaces, including dashboards of open decisions and graph representations of claim dependencies (similar to Figure 7) to get hold of the large amount of AI-generated derivations and code. However, we consider it an open question to the community what the human review surface *should* be for work such as this (Section 5.2). We note, with some discomfort, that the file management and tool calling underlying this workflow structurally exclude the human from the drafting process itself: every mathematical claim in the contracts was drafted by an AI agent, and the human role is to set the intent, accept and decide.

The workflow demonstrated that AI agents can speedily and cheaply do work that humans tend to skip: tedious bookkeeping, repeated adversarial review, exhaustive tests (over 8,900 at the time of writing) and careful accounting of numerical imprecision (Figure 5). Where AI agents still showed significant limits is in restraint and judgement. Process and output can grow quickly if not kept in check, and judgement calls made by AI can easily miss the human intent behind the project. We discuss this further in Section 5.2 and also raise the issue of correctness and trust in the result in Section 5.1.

3.3 Reviewing and extending the backend

PROOF BACKEND

A protocol within the scope of the existing modules needs no change to the CERTIQS-KEY backend: it is an input (instance and run parameters). A proof expert may wish to work on the backend itself, to review it, to extend it to protocols not yet in scope, or to change claims or code that affect the security claim.

Reviewing a module Each module has a contract. The contract contains every claim used in the module with its accompanying proofs. The module contracts can be found in `spec/modules/`. A claim lists all of its dependencies and assumptions (Figure 2). In addition, the repository contains a script that creates dependency graphs for any claim (similar to Figure 7). Each claim also carries its review status (draft, verified by an independent agent, accepted by a human, or implemented with reviewed code).

Extending the backend Adding a new proof reduction, device model or protocol class to the existing modules requires introducing new claims and code that implements them. This is done through the draft-and-verify loop of Section 3.2. The harness that runs such extensions is included in the open-sourced repository: the project's instructions (`AGENTS.md`), one file per agent role (such as drafter, verifier and implementer), the specified file formats (of claims, instances and run records), and the tools (Python scripts) for mechanical checks.

For example, a new feature is planned to allow for error bars on the detector efficiencies. This could likely be done through a characterisation submodule of the receiver and a new kind of reduction step, such as the combination of a flag-state squasher with a noise channel that

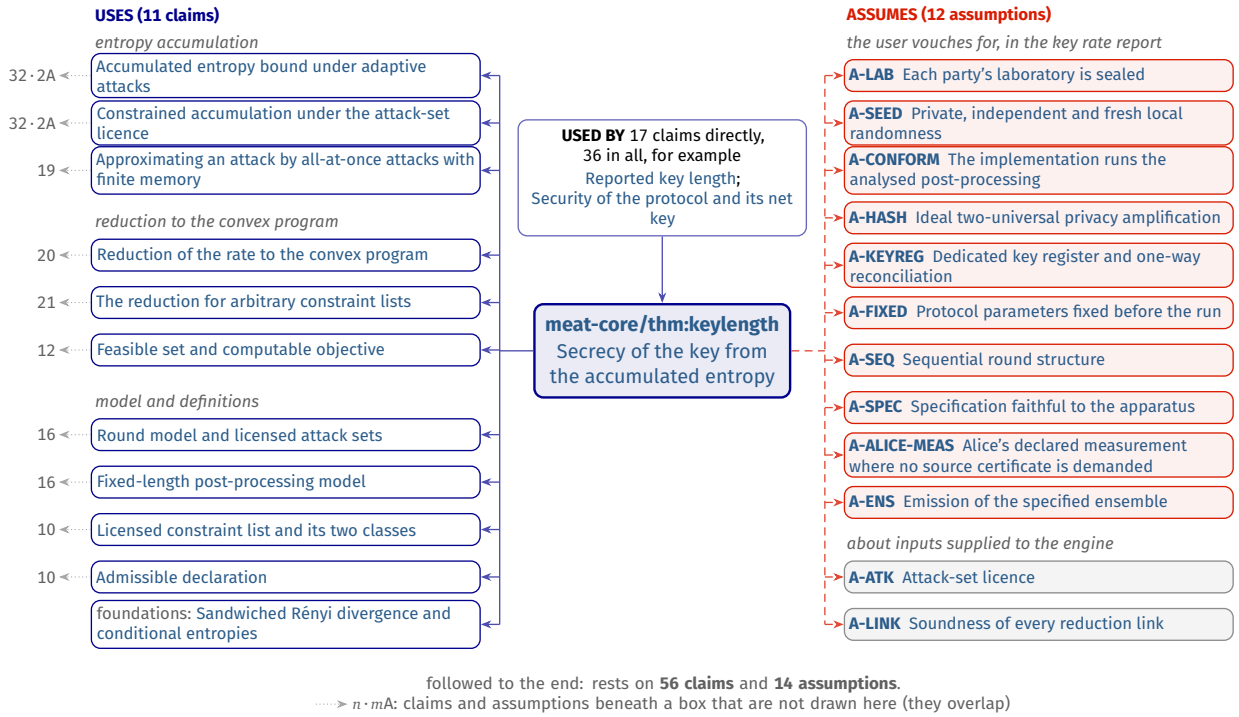


Figure 7: PROOF BACKEND One level of the claim graph around the key length theorem of Figure 2: the claims it uses (solid arrows) and the assumptions it names (dashed). Followed to the end, the graph shows what a reviewer (agent or human) of this theorem would have to read, and what is affected by any changes to it.

Wiesemann et al. [31, Lemmas 4–5] use for imperfectly characterised detectors. Likewise, a new squasher or source map cannot be supplied as an input to CERTIQS-KEY, it needs new claims in the backend.

We welcome anyone to clone the repository, make modifications or add features, and reach out to the authors about collaborations or pull requests.

4 Using CERTIQS-KEY

INTERFACE

CERTIQS-KEY provides a Python interface that calculates the key rate of a protocol instance for given run parameters (Section 3.1). However, this requires formalising the user's protocol in the input format CERTIQS-KEY accepts. We instead propose a different way to use CERTIQS-KEY, through coding agents: CERTIQS-KEY ships a Model Context Protocol (MCP) server, through which an agent such as Claude Code, Cursor or Codex can list the existing protocol instances, check and prepare a configuration, start a run and read its key rate report (setup instructions in its README). Like the Python interface, it is a thin layer over the run interface of Section 3.1: it passes inputs and results through. It only acts on the interfaces, and does not touch any of the security-critical backend code and mathematics.

Figure 8 shows the intended use flow. You describe your setup in plain words, pointing the AI agent to additional context as needed (such as documentation on the protocol and implementation, characterisation results, and click statistics). The agent maps the description onto one of the existing instances (ideal qubit BB84, BB84 with an optical receiver, decoy-state BB84 and BBM92) by setting its parameters or, when none fits, writes a protocol instance itself: a file that declares the protocol's parameters, source, receivers, public announcements,

key map and honest model, starting from the nearest instance as a template. CERTIQS-KEY checks the instance provided by the agent. If anything violates a checkable assumption or the input format, CERTIQS-KEY rejects the instance and provides a reason and the place of the offending entry so that the agent can correct it. CERTIQS-KEY then prepares the run (compiling the devices, building the acceptance test), runs it in the background and returns the key length with its key rate report.

Importantly, the agent's mapping is not trusted. All assumptions and everything the user or agent declared rather than CERTIQS-KEY derived, such as a measurement POVM written down directly instead of compiled through the receiver module, is marked as declared in the key rate report. The setup the run used is summarised at the top of the key rate report. With this design, the user's security-critical responsibility is to check that the assumptions and models the report lists hold for the actual devices.

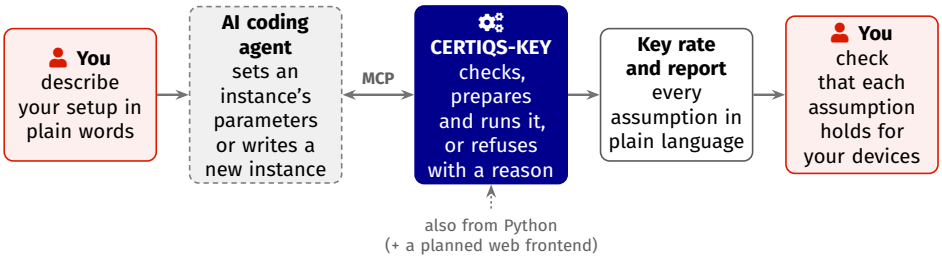


Figure 8: INTERFACE Intended use flow. Through the MCP server, an AI coding agent turns the user's plain-language description and provided documents into an instance. This instance is corrected through iteration with CERTIQS-KEY's acceptance criteria. Once an instance is accepted, CERTIQS-KEY calculates the key length and generates the key rate report for the user.

Example: BBM92 from a plain description

You Polarisation BBM92 with 10 dB loss, evenly split between both arms. Each receiver chooses its basis with a 56:44 beamsplitter; its polarising beamsplitters leak 0.1 % and 1 % into the wrong output; the H and D detectors have efficiency 0.8, V and A 0.7322; dark counts are 1 kHz with a 130 ps coincidence window; Bob's analyser is misaligned by $\sin^2\theta = 0.004$; the source visibility is 0.98; we collect about 10^6 coincidences. Our post-processing steps are described in `post-processing-steps.pdf`.

The agent finds BBM92 among the existing instances, reads its parameters and sets them per receiver, for example turning the dark-count rate and coincidence window into a dark-click probability per round. Such conversions are the agent's, and the report shows the values it used, for you to check. It also sets the number of rounds, e.g. $n = 1.7 \times 10^7$ emitted pairs for about 10^6 coincidences at this loss, and the leakage λ_{EC} of your error correction, which CERTIQS-KEY takes as an input.[5]

A run of this setup is used for the key rate plots in Figure 6. Its key rate is 0.017 bits per emitted pair, or about 0.29 bits per expected coincidence, with security parameter $\varepsilon \approx 1.9 \times 10^{-9}$. Error-correction leakage is deducted, taken as 1.1 times the Shannon limit of the expected statistics. The report of this run on the current engine lists 26 assumptions, 25 of which can break security if they fail. For example, the assumptions include that each receiver is trusted and exactly characterised, that each laboratory is sealed, and that the implementation runs the specified post-processing.
Figure 10, from a series of runs that add the example's realistic device values one at a time, shows the impact of various imperfections on the key rate.

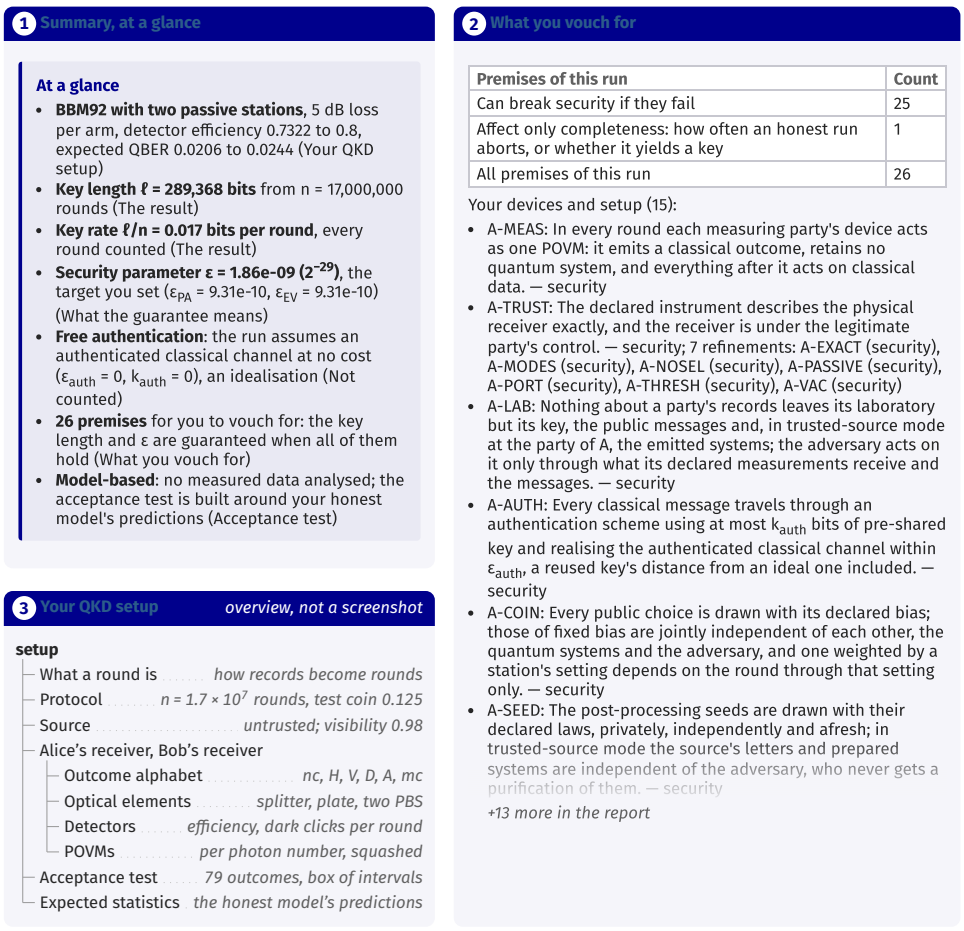


Figure 9: INTERFACE Excerpts from the key rate report of the example run. (1) The summary at its top: the setup, the key length and rate, the security parameter, and how many assumptions the user vouches for, with links to the detailed sections. (2) The assumptions, classified by whether their failure can break security. (3) An overview of the report's section on the setup the run used, from the round definition and the source to each receiver's optics, detectors and measurement, for the user to compare with the device. Each panel's header links to its section of the full report in the repository.

5 Discussion and vision

CERTIQS-KEY resulted from an experimental, AI-driven draft-and-verify workflow applied to QKD security proofs and numerical key rate calculation. The security proof and code being fully written by AI under supervision has raised various questions among the authors. In particular, it has led to discussions about:

1. What is required for security experts, industry partners, and certification authorities to trust AI-driven QKD security proofs? (Section 5.1)
2. If one accepts an AI-driven QKD security proof framework, what is the role of human expertise in QKD security proofs? (Section 5.2)

These are not questions that can be answered by the authors of this work alone (indeed, the authors may not even be in complete agreement). As such, a motivation for this whitepaper

[5]The device values are taken from published deployments: the visibility and the efficiency mismatch (the ratio of two detector efficiencies) from Wengerowsky et al. [32, p. 3], the H and D efficiency and the misalignment from Neumann et al. [33, pp. 5, 14], and the dark counts, coincidence window, basis split and leaks from Joshi et al. [34, pp. 2, 8]. The post-processing document is illustrative.

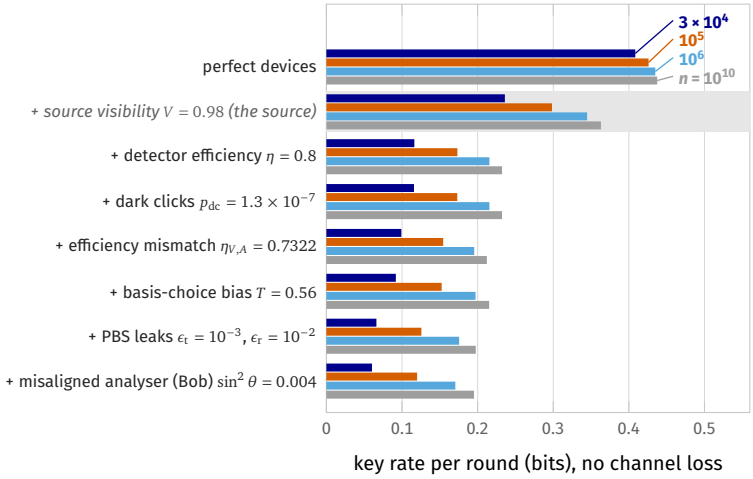


Figure 10: INTERFACE What each imperfection costs at a given number of rounds. The key rate per round at zero channel loss as the realistic device parameters of the example are added one at a time, top to bottom (each row keeps every imperfection above it). The key rate mostly decreases as more imperfections are added, with the relative effect much stronger for few rounds. The coloured bars represent different numbers of rounds, where we report the expected number of coincidences (rather than the actual number of rounds modelled) as this is what experimentalists typically report (3×10^4, 10^5, 10^6); the grey bar is a reference of $n = 10^{10}$ rounds. The difference between the grey bar of the top row and the dark blue bar of the bottom row demonstrates how critical it is to include imperfections and finite-size effects in the key length calculation. The shaded row belongs to the source, not the receiver (and affects the key rate only through the acceptance statistics).

and making CERTIQS-KEY public is to encourage the community to join these discussions. To that end, in the rest of this section, we present our current views on these topics.

We also believe that capabilities and features of CERTIQS-KEY have implications for some of the key challenges in the secure deployment of QKD. This includes security certification (Section 5.3) and QKD standardisation (Section 5.4), for which we present some ideas below. Overall, we hope that CERTIQS-KEY and the discussion in this whitepaper can contribute towards addressing the significant remaining challenges facing the deployment of QKD in high-security environments.

5.1 Trust and key rates

Key rates only matter if humans can trust that they are secure and applicable. The question then is how key rates are currently trusted, how we can (or cannot) trust the key rates produced by this framework, and what we can expect for the future.

●–○–○ STATUS QUO **Trust by human experts** A key rate has traditionally been trusted because experts wrote its proof and its code, and other experts reviewed the paper. Thus, trust was based on relying on a small group of people to have a deep understanding of the subject and others, like experimentalists or vendors, having confidence in these experts' research.

This approach is not without criticism, especially since few such experts exist globally and proofs largely continue to be produced within academia (rather than industry), stringing together a dispersed body of research papers, with few links to many of the established practices and concepts in 'classical' cybersecurity [1]. This makes the proofs hard to understand for researchers outside the community and prevents the wider diffusion of 'proof engineering' best practices. In addition, experimentalists trying to apply the

results of these security proofs would have to sift through the proofs to find and map (implicit) assumptions and modelling choices to their specific setup. In practice, this creates a gap between theoretical proofs and real implementations, complicating certification.

CERTIQS-KEY rests on two different sources of trust: trust 'by process' (today) and trust by formal verification (future outlook).

○–●–○ CERTIQS-KEY TODAY **Trust by process, relying on LLMs for correctness** This source of trust rests on the premise stated in the introduction: that publicly available frontier AI models are as good as, or better than, human experts at deriving correct proofs, finding mistakes, and implementing code to a specification. If the premise holds, then holding the agents to the workflow of a responsible human expert should give results at least as trustworthy as that expert's. That workflow comprises independent critical review, cross-checks against published results and limiting cases, explicit assumptions, precise citations of every imported result, and tested code (Section 3.2). It can be questioned whether adversarial review by AI truly constitutes 'independent review' in the same way human review processes separate author and reviewer. The CERTIQS-KEY workflow intends different model families for drafting and review, but in practice we often paired Opus with Fable (both Anthropic) and used Opus 5.5 for both once it became available. The review is not (yet) just a formality, with reviewing agents finding security-relevant issues during the development of CERTIQS-KEY. In one example, in September 2026, a contract claim specified privacy amplification with an ordinary two-universal hash family, whereas the Rényi leftover hashing lemma it used requires an ideal one. The key length was one bit too long, an error in the unsafe direction and one that, as Tupkary et al. [35, Rem. 6.1] note, is often missed in the literature. An independent verifier agent found it.

The premise is open to debate, and trust by process is not trust by understanding. Many parts of CERTIQS-KEY have not been read, let alone understood, by any of the authors of the whitepaper. Our own review has covered parts of the statements and assumptions along with some spot checks, and this has relied heavily on summaries written by AI. We therefore do not recommend CERTIQS-KEY for security-critical deployment at this stage.

Additionally, while one may trust the top-level results of the workflow, the steps to get to the main results might be compressed and, in places, convoluted beyond what even experts in the field can comfortably parse. Thus, for a human reader looking at the code or mathematical contracts (totalling over a thousand pages for CERTIQS-KEY), the implementation might not be intuitive or optimal. This complicates any attempt at a complete human review aimed at 'stacking' trust by human experts on top of trust by process.

○–○–● FUTURE **Trust by formal verification** In the future, the pinnacle of trust may increasingly rest on machine-checked proofs [36, 37]. A proof assistant such as Lean checks every step of a proof against its stated hypotheses, so that trust no longer depends on any reviewer, human or AI, following the argument.

In addition to formally verifying the proofs in Lean, code verification is essential. We need to trust that the code correctly implements the proof statements. Especially if used for evaluation purposes (Section 5.3), it is also important to ensure that the code is secure in the sense that QKD vendors' proprietary data stays private and that there are no hidden exploits that can change the reported key rate. On this front, there have been several promising frameworks (Cedar by AWS [38], Aeneas/SymCrypt by Microsoft [39], Veil [40]) and benchmarks (CLEVER [41], VeriBench [42]).

For CERTIQS-KEY, formal verification of the MEAT core module would be the highest priority, and it should leverage the split of the a posteriori certification in Section 3.1. The convex solver can remain untrusted: it merely searches for a candidate attack state, the point at which the bound is evaluated, and hands it over with the data needed to check it. A smaller, formally verified checker recomputes the bound at that candidate and either derives the key length or rejects. In addition, device reductions, mapping realistic device models to numerically tractable ones, should be formally verified. This split is shown in Figure 11.

Actually implementing such a formal verification for CERTIQS-KEY will require further research and planning and has, currently, only been preliminarily explored. CERTIQS-KEY's claim files are prepared for this route (Section 3.1), but no part of them has been formalised yet.

Formal verification does not remove trust completely. Rather (trusting the proof assistant itself), it confines it to three places: the definitions and the top-level statement, the imported theorems such as MEAT [19] cited in the contracts (taken as axioms until they are themselves formalised), and the device model. The closest precedents we know of are two recent machine-checked proofs of composable finite-key security of qubit BB84, one in Lean [43] and one in Rocq [44].

5.2 The role of human expertise in QKD security proofs

●○○ STATUS QUO **Expert derivation and review** As mentioned above, deriving correct and practical QKD security proofs has so far been limited to a relatively small community. The derivation, careful write-up, and review of mathematical proofs for key rate expressions, as well as understanding the definitions, mathematical tools, and computational tools that go into them were what took the most expert work hours. In the development of CERTIQS-KEY, it is exactly these steps that were largely handed over to agentic AI.

○●○ CERTIQS-KEY TODAY **Design, guide and decide** Prior to developing CERTIQS-KEY, we spent time building our human understanding of security proofs: understanding the structure of security proofs, the mathematical and numerical tools, device modelling and working through this process end to end for specific examples. In the development of CERTIQS-KEY, this human expertise then played largely the role of providing initial designs, structural requirements on the workflow and presentation, and inputs on *what* should be derived and implemented. For *how* to implement it, human input sometimes took the form of internal notes, literature pointers, or was just left entirely up to the AI. Agentic AI formulated all claims with their proofs. It then collected them with definitions, assumptions, and implementation requirements into contract documents. These contracts were then implemented into the numerics code.

Human approval and choices entered the development process continuously, but actual human review was uneven and often ad hoc. Currently, detailed understanding among the authors is largely limited to parts that closely follow results we previously derived ourselves. These results were special cases that were then generalised within CERTIQS-KEY.

○○● FUTURE **What remains human**

Based on the experience developing CERTIQS-KEY, we expect the human role in applied security proofs to increasingly move towards asserting that the stated assumptions and top-level claims map to the actual protocol and devices. Derivations, helper lemmas

and implementation would in this view be delegated increasingly to AI agents and formal verification.

At a high level, at least three things remain for humans. The first is the top-level statement and its definitions, since even a formally verified proof that compiles can still prove a weaker statement than intended. Ben-Shahar et al. [43, Secs. I–II] accordingly reviewed definitions and theorem statements rather than proof bodies, and that human review caught an AI-written protocol whose key was a hash of both parties' strings, despite neither party being able to hold both, and which was therefore not viable in practice. The second is whether the declared model and its assumptions fit the actual devices. For practical QKD, a proof is meaningless if it does not map to implemented hardware. This includes finding good models in the first place (e.g. for source and receiver correlations between rounds or for attacks such as Trojan-horse attacks) and connecting them to practical tests on real QKD devices. The third is judgement: knowing what the work is for, how it is meant to be used and what trade-offs are acceptable.

These three points are not an exhaustive list. For example, currently (October 2026), running agentic workflows itself may increasingly become a matter of expertise, as may developing human-friendly review surfaces (and methods to ensure their reliability). Running AI-driven workflows requires carefully stating any intent that agents cannot infer, prioritising and navigating within an overwhelming amount of AI-generated output, and noticing when 'process bloat' goes off track.

The question of where human expertise is necessary from a technical perspective must also be separated from the question of where human expertise should be required from a societal and accountability perspective. How much human review is enough to establish institutional and societal trust in the resulting key rates in production systems? This is a question the community has to answer over time. As a starting point, we propose three layers.

- *Must be reviewed by humans in detail*: the security statement and its definitions and assumptions, the modelling of the devices, and every imported external result that affects the resulting key rate.
- *High-level understanding by humans*: the overall logical flow of the argument, that is, important intermediate claims and how these claims chain into the security statement. A random sample of proofs and implementations fully read can complement this layer. It can reveal structural problems that statements alone hide and reveal issues in the workflow and design architecture itself.
- *Delegated*: helper lemmas, proofs in full, resulting requirements on the implementation and the code, to independent AI review, tests and formal verification.

5.3 Implications for security certification

Security certification of QKD systems by national certification bodies for high-security use is one of the most critical contemporary challenges within the QKD community. European security agencies judge QKD 'not yet sufficiently mature from a security perspective', naming the lack of comprehensive security proofs for practically relevant protocols and of recognised evaluation criteria and methodologies among the reasons [1, pp. 1, 4–5].

●○○ STATUS QUO **Three layers of certification** There are three layers that should be distinguished in terms of certification and the responsible entities:

1. *An approved security proof and its key rate calculation.* Common Criteria keeps the

proof outside the evaluation of the product itself. Under the protection profile ETSI GS QKD 016, the developer delivers 'the complete, correct, and comprehensible security proof', which 'shall be approved by the responsible certification body', possibly drawing on 'the opinion of a reputable group, such as a standards developing organization' [9, Sec. 9.1]. Evaluating such a proof by hand, and the numerical key rate computed from it, is a heavy task for which cybersecurity agencies generally lack the relevant expertise, and it has rarely been carried out in practice.[6]

2. *The QKD system against the proof.* An evaluation laboratory checks whether and how the implementation ensures the assumptions of the security proof, and whether and how limitations of the model or imperfections of the implementation impact the key [9, Sec. 9.1]. This includes hardware as well as the post-processing (e.g. [9, FCS_QKD.1.4–1.6]). ISO/IEC 23837 sets out such an evaluation under Common Criteria, with test methods for the transmitter and receiver modules [10, 11].

3. *The QKD network.* In a network, each QKD link becomes part of a system of trusted nodes, key relay, key management, and control and management, each with security requirements of its own [45, Secs. 4–7]. Security proofs assume some aspects of the network, such as sealed laboratories, the trusted nodes of the network, and an authenticated classical channel.

A quantified connection between the first two layers is largely missing. Existing standards on evaluation activities output pass or fail decisions on whether a device meets its requirements, but do not provide actual quantified thresholds for these decisions [11]. The evaluation under [9, Sec. 9.1] must determine whether such limitations and imperfections impact the key, but it is 'not required to determine how such effects affect the security parameters': the assessment may remain qualitative. It therefore largely remains open how to establish a link between quantitative characterisations subject to measurement uncertainty – such as component characterisations (e.g. detection efficiencies) or system-level tests such as those in [11] – to the secure key rate.

CERTIQS-KEY TODAY **Where CERTIQS-KEY comes in** CERTIQS-KEY is relevant to the first two layers and partially addresses the gap between them from the side of the proof. To evaluate if the security proof applies to the stated implementation, device properties such as detection efficiencies and dark-count probabilities enter the security proof as declared parameters, so the effect of a measured imperfection on the key rate is computed rather than resting on qualitative judgements (Figure 10 shows this for the receiver). Every run's report lists the assumptions its key rate rests on, which provides a starting point for the 'detailed mapping of the assumptions of the security proof to the implementation' that the developer must deliver [9, Sec. 9.1]. That same list of assumptions is where the network enters: assumptions such as sealed laboratories and a post-processing procedure must name the required inputs the other layers will provide. That said, the current implementation of CERTIQS-KEY is still limited: parameters are taken as exact values, so a measurement's uncertainty does not yet enter the proof (Section 3.1), and CERTIQS-KEY

[6] There have been two recent efforts which aimed at certification for self-contained proofs of decoy-state BB84. In Japan, the Quantum Forum published a protocol description and security proof with an analytic key rate as a reference for Common Criteria certification of QKD modules, reviewed by an expert committee [46, 47]. Tupkary et al. [35] wrote a rigorous and complete proof based on MEAT (which provided significant inspiration and intellectual input to CERTIQS-KEY) within a project commissioned by the German Federal Ministry of Finance, with feedback from the BSI on its drafts [35, Acknowledgments].

is itself not certified (nor do we recommend its current early release for security-critical deployment (Section 5.1)).

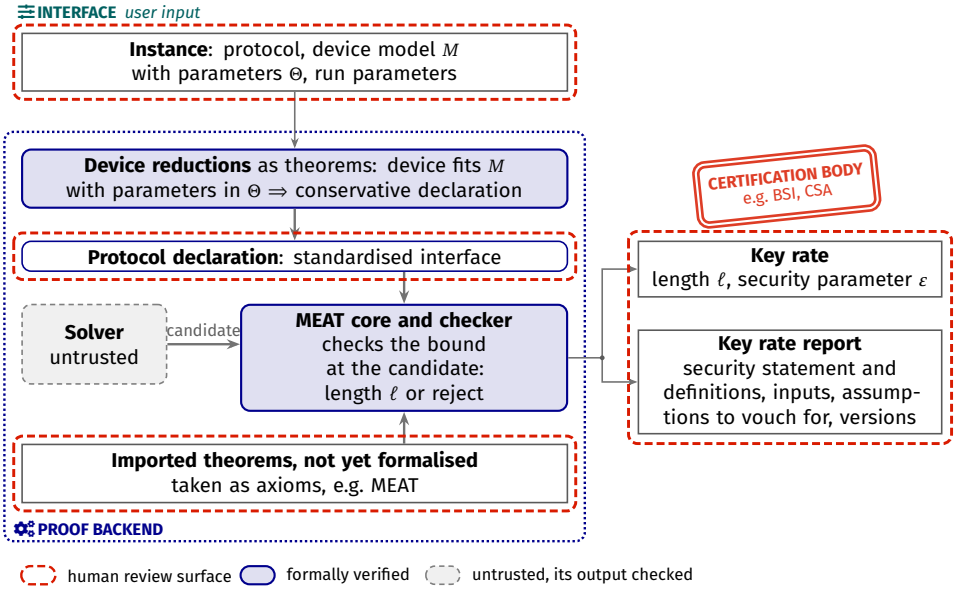


Figure 11: A future certification concept built on the architecture of Figure 1. Standardised interfaces carry the protocol declaration in and the key rate and its report out. Everything between the device model and the key length, except the solver (which may remain untrusted) and the imported theorems not yet formalised, is formally verified. The solver only proposes a candidate, at which the bound is checked, so a poor solver can lower the key length but not make it insecure. The red dashed boxes are what humans still review: the device model and its parameters, the standard that defines the protocol declaration, the imported theorems not yet formalised, and the report with the security statement the key satisfies. A certification body, such as the German Federal Office for Information Security (BSI) or Singapore's Cyber Security Agency (CSA), could accept the key rate and its report as evidence when certifying a QKD system.

FUTURE **A certification concept** Figure 11 sketches how the architecture of CERTIQS-KEY could lighten certification challenges and possibly allow for the certification of a key rate calculation software that can support generic entanglement-based and prepare-and-measure discrete-variable QKD protocols. With the formal verification outlined in Section 5.1, the human review surface would shrink to the red dashed boxes of Figure 11, cutting out what today are some of the largest but at the same time most inaccessible aspects of security proofs. Following some form of independent review for the security claims, device models and proof assistant interfaces, the numerical backend could be frozen to a *certified* version. Whether this review surface is indeed small enough for practical certification of a key rate engine is an open question that we hope to explore with certification bodies and standards developers.

5.4 Standardisation

Closely linked to certification, internationally adopted standards form an important prerequisite to the commercial adoption of QKD. We believe this will be even more important as AI-generated content increases in prevalence.[7]

[7] When agents derive proofs and write code, standards agreed by humans can set the terms and limits within which they work. The quality and the choice of standards then matter more, and assessing them remains an

●○○ STATUS QUO **No international protocol standard** To date, no international standard specifies a QKD protocol together with a security proof for it. European security agencies note that, to the best of their knowledge, 'no QKD protocol has undergone such a standardisation process' [1, p. 4]. Many standardisation efforts have instead worked around the protocol, on networks, interfaces, components and the evaluation of QKD modules.

One can broadly distinguish two approaches to QKD protocol standards:

Top-down: a framework that most protocols fit. ITU-T X.1711 divides a protocol into a quantum communication stage and a key distillation stage and describes the steps of each [13, Cl. 7]. A framework fixes structure and vocabulary, but it is too general for a security proof.

Bottom-up: one concrete protocol with one self-contained proof. In Japan, the Quantum Forum published a protocol description and security proof of decoy-state BB84 as a reference for Common Criteria evaluation [46, 47], together with a supporting document that maps the assumptions of security proofs to testable device characteristics [48, Table 3-1]. Tupkary et al. [35] wrote a complete proof of decoy-state BB84, another potential path to such bottom-up standardisation. National standards have gone furthest in China, where GM/T 0108-2021 specifies decoy-state BB84 stage by stage, from the signal intensities and their tolerances to error verification and privacy amplification [49], and GM/T 0114-2021 and mboxYD/T 3835.1-2021 specify how products and systems are tested against such a specification [50, 51]. However, these standards, while being specific in their implementation and test requirements, provide no security proof and state a key rate formula only in a non-normative annex (GM/T 0108, without proof). In general, a bottom-up document binds its proof to one protocol and one device model, reducing vendor diversity and flexibility to future changes: every variant would need a new proof and a new review.

○●○ CERTIQS-KEY TODAY **A protocol format by necessity** The modular architecture of CERTIQS-KEY combined with treating protocol structure and device specifications as inputs forced a protocol format: the protocol declaration, processed by protocol-specific modules, and then handed to the protocol-agnostic MEAT core (Section 3.1, Figure 12). Its structure is that of the generic QKD protocol of Tupkary et al. [35, Sec. 4.1]. CERTIQS-KEY's round model and its fixed-length post-processing are taken from that protocol. The protocol declaration itself is a machine-readable format with some checkable conditions (e.g. format and logic checks on declared states, measurement operators and classical maps). The declaration admits both prepare-and-measure and entanglement-based protocols. Its rounds and its post-processing correspond to the quantum communication and key distillation stages of ITU-T X.1711 (Figure 3).

○○● FUTURE **A generic protocol format and a common language** Below we share some ideas for standardisation directions we believe are promising following the development of CERTIQS-KEY.

A generic protocol format. Between a framework too general for a proof and a proof bound to one protocol, a standard could fix a protocol format for a class of protocols: as

important task for human experts. At the same time, AI makes standards easier to follow: complying with a standard by hand is tedious, whereas an agent given the standard in its context can easily be held to its terminology.

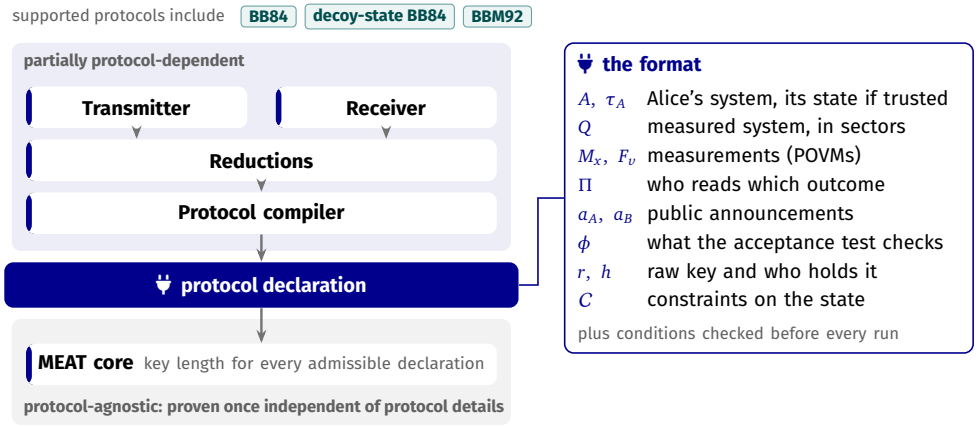


Figure 12: The protocol declaration, the interface CERTIQS-KEY's architecture forced. The partially protocol- and device-specific modules turn a protocol and its devices into one declaration format (right side). The MEAT core module turns any declaration that meets the format's conditions into a key length, so its proof, done once, covers every protocol written in the format.

general as possible, and as specific as needed to serve as the input of a single numerical key rate engine. A protocol written in the format would then be covered by the engine's proof, under the engine's assumptions, and a new variant would only need a new declaration rather than a new proof. In the certification concept of Figure 11, such a standard defines the protocol declaration box: it is one of the pieces humans would still review, and every protocol written in it reaches the formally verified core through it. CERTIQS-KEY makes an attempt at such a protocol format. CERTIQS-KEY already includes protocol instances for ideal and optical BB84, decoy-state BB84 and the entanglement-based BBM92, all compatible with the protocol declaration format of CERTIQS-KEY (Section 3.1). Based on this we believe the format is promising for discrete-variable entanglement-based and prepare-and-measure protocols more broadly. How far such a format extends, for instance to measurement-assisted or continuous-variable protocols, and how much should be specified in standards, remains for future work.

A common language between proofs and tests. The gap described in Section 5.3 is also one of language. A security proof states assumptions and parameters, a test laboratory measures devices and reports pass or fail, and a quantitative connection between the two is largely missing. Today's evaluation documents do not require this connection [9, Sec. 9.1], although ETSI GS QKD 005 envisaged it early on [52, Cl. 4.3]. A reporting standard could fix what must accompany a key rate: the security claim with its ε, the assumptions, the modelling choices, and the parameters. It could also standardise requirements on how assumptions are tested and how parameters are measured. With a certified key rate engine (Figure 11), measured values could then be turned directly into a key length, without new proof work for each new device or protocol logic the engine can model. CERTIQS-KEY's key rate report provides a concrete attempt in this direction. It lists for every run the security claim, the assumptions the key rate rests on, the device model and the values used (Section 4), but it does not link assumptions or values to practical tests. The current key rate report likely leaves much to be desired, but we hope it can serve as a starting point for a concrete discussion between security proof experts and test laboratories on what such a report should contain.

5.5 Limitations

CERTIQS-KEY and the results in this whitepaper are preliminary. Significant human review of the contracts and code is still outstanding, the contracts are not yet written for human readers, and no proof has undergone formal verification (Section 5.1). On the device side, parameters such as detection efficiencies and dark-count probabilities are taken as exact values rather than ranges, an assumption every key rate report lists and a limitation we plan to address. The same device model applies in every round, so correlations between rounds, such as detector dead time or afterpulsing, are not covered, and the receiver module is currently limited to passive linear optics with threshold detectors. On the protocol side, CERTIQS-KEY computes fixed-length key rates for discrete-variable entanglement-based and prepare-and-measure protocols. Variable-length key rates and measurement-device-independent, twin-field and continuous-variable protocols have so far not been investigated.

5.6 Acknowledgement of prior work

CERTIQS-KEY builds on many results of the QKD security community. Here we name those it draws on most directly, which have been critical references throughout the development. Its security argument rests on the marginal-constrained entropy accumulation theorem of Arqand and Tan [19], the Rényi security framework of Kamin et al. [3] and the composable security definitions of Portmann and Renner [12]. The protocol structure, from the post-processing steps to source replacement and decoy-state tagging, follows the decoy-state BB84 proof of Tupkary et al. [35]. The receiver module's description of passive optics builds on Kamin and Lütkenhaus [21], and the reductions on the flag-state squasher of Zhang et al. [22] and its weight-preserving variant by Nahar et al. [53]. The numerical approach, a near-optimal attack turned into a reliable lower bound, is that of Winick et al. [2]. Beyond individual results, the structure and approach of OpenQKDSecurity [4] were a large inspiration, and the published rates of Navarro et al. [6] served as cross-checks at shared parameter point.

CERTIQS-KEY also relies on open-source software. QICS [18] is used as a semidefinite program solver and its quantum key distribution cone is used in one stage of CERTIQS-KEY's convex solver. Local optimisers of SciPy [54] also drive two of the implemented solver paths. The ball arithmetic of Arb, through python-flint [27, 55], implements the certified tier. NumPy [56] implements the floating-point linear algebra.

AI assistance

AI agents built on, among others, Claude Opus 5, Claude Opus 5.5 and Claude Fable (Anthropic), and on GPT-5.6 Sol, GPT-6 Astra and GPT-6.1 Sol (OpenAI) drafted, reviewed and implemented the contracts, code and tests of CERTIQS-KEY in the workflow of Section 3.2, and assisted in drafting this whitepaper. The authors have fully reviewed and take responsibility for the content of this whitepaper. (The review status of CERTIQS-KEY itself is described in Section 5.1.)

Acknowledgements

We are grateful to Ernest Tan for his feedback and input, which through many discussions shaped CERTIQS-KEY. We thank Hao Qin for helpful discussions and feedback, especially on standardisation-related aspects and practical perspectives. As part of the Quantum Security and Resilience for Emerging Technologies (QUASAR-CREATE) project, this research is supported by the National Research Foundation, Prime Minister's Office, Singapore under its Campus for Research Excellence and Technological Enterprise (CREATE) programme.

This project is supported by the National Research Foundation, Singapore under the NRF Postdoctoral award.

References

[1] BSI et al. *Position Paper on Quantum Key Distribution*. Position paper. 26th Jan. 2024. URL: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Crypto/Quantum_Positionspapier.html.

[2] Adam Winick, Norbert Lütkenhaus and Patrick J. Coles. 'Reliable numerical key rates for quantum key distribution'. In: *Quantum 2, 77* (2018). DOI: 10.22331/q-2018-07-26-77. arXiv: 1710.05511v2.

[3] Lars Kamin, John Burniston and Ernest Y.-Z. Tan. *Rényi security framework against coherent attacks applied to decoy-state QKD*. 2025. arXiv: 2504.12248v2.

[4] John Burniston et al. *Open QKD Security*. Version 2.0.2. 2nd Dec. 2024. DOI: 10.5281/zenodo.14262569.

[5] Andrés González Lorente et al. 'Quantum key distribution rates from non-symmetric conic optimization'. In: *Quantum* 9 (2025), p. 1657. DOI: 10.22331/q-2025-03-10-1657. arXiv: 2407.00152.

[6] Mariana Navarro et al. *Finite-size quantum key distribution rates from Rényi entropies using conic optimization*. 2025. arXiv: 2511.10584v3.

[7] Devashish Tupkary et al. *Security proofs for practical QKD: variations, techniques, gaps, and limitations*. 2025. arXiv: 2502.10340v3.

[8] John Burniston, Scott Johnstun and Lars Kamin. *OpenQKDSecurity Version 2 User Guide*. 2024. URL: https://github.com/Optical-Quantum-Communication-Theory/openQKDsecurity/blob/main/OpenQKDSecurity_Version_2_User_Guide.pdf.

[9] ETSI Industry Specification Group QKD. *ETSI GS QKD 016 V2.1.1 (2024-01): Quantum Key Distribution (QKD); Common Criteria Protection Profile - Pair of Prepare and Measure Quantum Key Distribution Modules*. 2024.

[10] ISO/IEC JTC 1/SC 27. *Information security – Security requirements, test and evaluation methods for quantum key distribution – Part 1: Requirements*. International Standard ISO/IEC 23837-1:2023. International Organization for Standardization, Aug. 2023. URL: https://www.iso.org/standard/77097.html.

[11] ISO/IEC JTC 1/SC 27. *Information security – Security requirements, test and evaluation methods for quantum key distribution – Part 2: Evaluation and testing methods*. International Standard ISO/IEC 23837-2:2023. International Organization for Standardization, Sept. 2023. URL: https://www.iso.org/standard/77309.html.

[12] Christopher Portmann and Renato Renner. 'Security in quantum cryptography'. In: *Rev. Mod. Phys. 94, 025008* (2022). DOI: 10.1103/RevModPhys.94.025008. arXiv: 2102.00021v2.

[13] ITU-T Study Group 17. *Recommendation ITU-T X.1711 (03/2026): Framework of quantum key distribution (QKD) protocols in QKD networks*. 2026.

[14] Devashish Tupkary et al. 'Phase error rate estimation in QKD with imperfect detectors'. In: *Quantum* 9 (Dec. 2025), p. 1937. DOI: 10.22331/q-2025-12-11-1937.

[15] Patrick J. Coles, Eric M. Metodiev and Norbert Lütkenhaus. 'Numerical approach for unstructured quantum key distribution'. In: *Nature Communications* 7.1 (May 2016), p. 11712. DOI: 10.1038/ncomms11712.

[16] Jie Lin, Twesh Upadhyaya and Norbert Lütkenhaus. 'Asymptotic security analysis of discrete-modulated continuous-variable quantum key distribution'. In: *Phys. Rev. X 9, 041064* (2019). DOI: 10.1103/PhysRevX.9.041064. arXiv: 1905.10896v2.

[17] Ian George, Jie Lin and Norbert Lütkenhaus. 'Numerical calculations of the finite key rate for general quantum key distribution protocols'. In: *Physical Review Research* 3.1 (Mar. 2021), p. 013274. DOI: 10.1103/physrevresearch.3.013274.

[18] Kerry He, James Saunderson and Hamza Fawzi. *QICS: Quantum Information Conic Solver*. 2025. arXiv: 2410.17803 [math.OC]. URL: https://arxiv.org/abs/2410.17803.

[19] Amir Arqand and Ernest Y.-Z. Tan. *Marginal-constrained entropy accumulation theorem*. 2025. arXiv: 2502.02563v4.

[20] Patrick Massot. *leanblueprint: a plasTeX plugin to build formalization blueprints*. 2021. URL: https://github.com/PatrickMassot/leanblueprint.

[21] Lars Kamin and Norbert Lütkenhaus. 'Improved decoy-state and flag-state squashing methods'. In: *Phys. Rev. Research 6, 043223* (2024). DOI: 10.1103/PhysRevResearch.6.043223. arXiv: 2405.05069v2.

[22] Yanbao Zhang et al. 'Security proof of practical quantum key distribution with detection-efficiency mismatch'. In: *Phys. Rev. Research 3, 013076* (2021). DOI: 10.1103/PhysRevResearch.3.013076. arXiv: 2004.04383v2.

[23] Hao Hu et al. 'Robust interior point method for quantum key distribution rate computation'. In: *Quantum* 6 (2022), p. 792. DOI: 10.22331/q-2022-09-08-792.

[24] Bence Temesi et al. *Efficient Computation of QKD Key Rates without Semidefinite Programming*. Version 2. 2026. arXiv: 2608.23285 [quant-ph].

[25] Siegfried M. Rump. 'Verification methods: Rigorous results using floating-point arithmetic'. In: *Acta Numerica* 19 (May 2010), pp. 287–449. DOI: 10.1017/S096249291000005X.

[26] Christian Jansson, Denis Chaykin and Christian Keil. 'Rigorous Error Bounds for the Optimal Value in Semidefinite Programming'. In: *SIAM Journal on Numerical Analysis* 46.1 (2007), pp. 180–200. DOI: 10.1137/050622870.

[27] Fredrik Johansson. 'Arb: Efficient Arbitrary-Precision Midpoint-Radius Interval Arithmetic'. In: *IEEE Transactions on Computers* 66.8 (Aug. 2017), pp. 1281–1292. DOI: 10.1109/TC.2017.2690633.

[28] Marguerite Frank and Philip Wolfe. 'An algorithm for quadratic programming'. In: *Naval Research Logistics Quarterly* 3.1-2 (Mar. 1956), pp. 95–110. DOI: 10.1002/nav.3800030109.

[29] Anthropic. *Formalizing Fermat's Last Theorem*. 4th Sept. 2026. URL: https://www.anthropic.com/research/formalizing-fermats-last-theorem.

[30] Shuze Chen et al. *Prove2Me: An Open Collaborative Platform for Scaling Math Formalization*. 2026. arXiv: 2608.28433v2 [cs.AI].

[31] Jerome Wiesemann et al. *Security framework for practical quantum key distribution with imperfect devices*. 2026. arXiv: 2609.15790v1.

[32] Sören Wengerowsky et al. 'In-field entanglement distribution over a 96 km-long submarine optical fibre'. In: *PNAS 116 (14), 6684-6688* (2019). DOI: 10.1073/pnas.1818752116. arXiv: 1803.00583v1.

[33] Sebastian Philipp Neumann et al. 'Continuous entanglement distribution over a transnational 248 km fibre link'. In: *Nature Communications 13, 6134* (2022). DOI: 10.1038/s41467-022-33919-0. arXiv: 2203.12417v1.

[34] Siddarth Koduru Joshi et al. 'A trusted-node-free eight-user metropolitan quantum communication network'. In: *Science Advances 6, no. 36 (2020): eaba0959* (2020). DOI: 10.1126/sciadv.aba0959. arXiv: 1907.08229v4.

[35] Devashish Tupkary et al. *A rigorous and complete security proof of decoy-state BB84 quantum key distribution*. 2026. arXiv: 2601.18035v1.

[36] Keane Lucas et al. *LLMs are making formal software verification feasible at a much larger scale*. Anthropic. 22nd Oct. 2026. URL: https://www.anthropic.com/research/formal-verification.

[37] Leonardo de Moura. *When AI Writes the World's Software, Who Verifies It?* Leonardo de Moura's Blog. Accessed: 2026-10-23. Feb. 2026. URL: https://leodemoura.github.io/blog/2026-2-28-when-ai-writes-the-worlds-software-who-verifies-it/.

[38] Kesha Hietala and Emina Torlak. *Lean Into Verified Software Development*. AWS Open Source Blog. Accessed: 2026-10-23. Apr. 2024. URL: https://aws.amazon.com/blogs/opensource/lean-into-verified-software-development/.

[39] Son Ho et al. *Scaling Verification of Cryptographic Software with Aeneas, Rust, and Lean*. Sept. 2026. DOI: 10.48550/arXiv.2609.15648. arXiv: 2609.15648 [cs.CR].

[40] George Pîrlea. *Veil: Multi-Modal Verification of Distributed Protocols*. https://lean-lang.org/use-cases/veil/. Lean Lang. Accessed: 2026-10-23.

[41] Amitayush Thakur et al. *CLEVER: A Curated Benchmark for Formally Verified Code Generation*. 2025. arXiv: 2505.13938 [cs.LG]. URL: https://arxiv.org/abs/2505.13938.

[42] Brando Miranda et al. *VeriBench: End-to-End Formal Verification Benchmark for AI Code Generation in Lean 4*. 2026. URL: https://openreview.net/forum?id=P7NUVF6wo4.

[43] Maor Ben-Shahar et al. *Quantum Cryptography and Autoformalization in Lean*. Version 1. 12th Oct. 2026. arXiv: 2610.17286 [quant-ph].

[44] Ruixuan Deng et al. *FVQKD: A Formal Verification of Quantum Key Distribution Security in Rocq*. Version 1. 22nd Oct. 2026. arXiv: 2610.30645 [quant-ph].

[45] Info-communications Media Development Authority. *Reference Specification for Secure Deployment of Quantum Key Distribution Networks*. Reference Specification IMDA RS SecQKDN, Issue 1. Singapore: Info-communications Media Development Authority, July 2026. URL: https://www.imda.gov.sg/assets/ed165484-cf19-4367-8581-7995360bff30.pdf.

[46] Akihiro Mizutani et al. *Protocol and Security Proof of Decoy-state BB84 Quantum Key Distribution*. Technical document QF-TD-QKD-2025-003, Version 1.0. Quantum Key Distribution Technology Promotion Committee, Quantum Forum, May 2025. URL: https://qforum.org/assets/documents/20250514-PSPD(1.0).pdf.

[47] Akihiro Mizutani, Toshihiko Sasaki and Go Kato. *Protocol-level description and self-contained security proof of decoy-state BB84 QKD protocol*. Version 3. 2025. arXiv: 2504.20417 [quant-ph].

[48] Quantum Key Distribution Technology Promotion Committee. *Supporting Document, Mandatory Technical Document: Evaluation Method for Protection Profile for Prepare and Measure Quantum Key Distribution Modules*. Technical document QF-TD-QKD-2025-002, Version 1.0. Quantum Forum, May 2025. URL: https://qforum.org/assets/documents/20250513-SD(1.0)EnJp.pdf.

[49] State Cryptography Administration. *Decoy-state BB84 quantum key distribution product technology specification*. Cryptography Industry Standard GM/T 0108-2021. In Chinese. State Cryptography Administration of China, 18th Oct. 2021.

[50] State Cryptography Administration. *Decoy-state BB84 quantum key distribution product testing specification*. Cryptography Industry Standard GM/T 0114-2021. In Chinese. State Cryptography Administration of China, Oct. 2021.

[51] Ministry of Industry and Information Technology. *Test methods for quantum key distribution (QKD) system – Part 1: Decoy state BB84 protocol QKD system*. Telecommunication Industry Standard YD/T 3835.1-2021. In Chinese. Ministry of Industry and Information Technology of China, 5th Mar. 2021.

[52] ETSI Industry Specification Group QKD. *ETSI GS QKD 005 V1.1.1 (2010-12): Quantum Key Distribution (QKD); Security Proofs*. 2010.

[53] Shlok Nahar et al. 'Postselection technique for optical Quantum Key Distribution with improved de Finetti reductions'. In: *PRX Quantum 5, 040315* (2024). DOI: 10.1103/PRXQuantum.5.040315. arXiv: 2403.11851v3.

[54] Pauli Virtanen et al. 'SciPy 1.0: fundamental algorithms for scientific computing in Python'. In: *Nature Methods* 17 (Feb. 2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[55] Fredrik Johansson and Oscar Benjamin. *python-flint: Bindings for FLINT*. Version 0.9.0. 3rd July 2026. URL: https://github.com/flintlib/python-flint.

[56] Charles R. Harris et al. 'Array programming with NumPy'. In: *Nature* 585 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.